

# Protocol Emulator ASIC — Architecture Planning Brief

### *Problem framing, proposed cores, protocol mapping, and architecture questions*

Prepared as a starting brief for architectural exploration / Codex planning

**Core thesis.** The chip should not be a collection of fixed UART/SPI/I<sup>2</sup>C/USB/Ethernet controllers. It should be a programmable protocol machine: a small control core paired with protocol-agnostic timing, serialization, sampling, pin, line-coding, and CRC/LFSR primitives. The strongest architecture is likely a two-level design in which software controls protocol semantics while specialized but generic micro-engines perform cycle-critical wire operations.

## 1. Background and Problem Statement

The competition asks for an open-source, general-purpose protocol emulator ASIC. Jane Street describes the target as a tiny CPU whose instruction set is designed for reading pins, writing pins, counting cycles, and meeting protocol timing precisely enough that protocols can be implemented in firmware rather than as fixed logic.

The required baseline is UART, SPI, and I<sup>2</sup>C. Stretch goals include low-speed USB and 10 Mbit Ethernet, with JTAG, SWD, PS/2, and CAN also suggested as interesting targets. The central requirement is reprogrammability: the design should be able to support protocols that were not hardwired before fabrication, subject to the chip's timing and I/O limits.

This creates a specific architectural tension. A conventional CPU plus GPIO is highly programmable but inefficient and slow at exact wire timing. A dedicated protocol controller is fast and compact, but it violates the spirit of protocol emulation because the protocol is embedded in fixed logic. The design problem is therefore to identify the reusable hardware primitives that many protocols share, accelerate those primitives, and leave protocol semantics under program control.

Too generic

Too fixed

Tiny CPU + GPIO <---- desired design space ----> UART/SPI/I2C/USB/MAC blocks

- + flexible, but slow

- + fast, but protocol-specific

Target: programmable control + generic cycle-critical I/O primitives

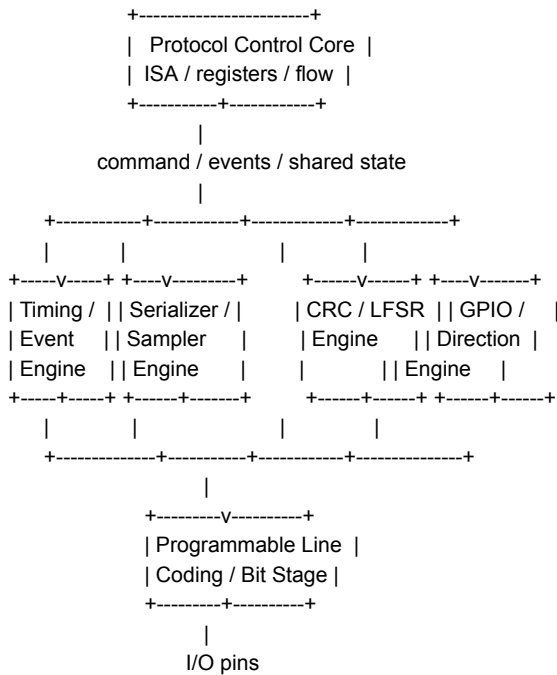
## 2. Architectural Principle: Accelerate Mechanisms, Not Protocols

The most useful design rule is: **a hardware block is a good candidate if it is naturally reusable by several unrelated protocols.**

| Candidate block                    | Classification                   | Fits emulator goal?               |
|------------------------------------|----------------------------------|-----------------------------------|
| UART transmitter                   | Protocol-specific                | No                                |
| SPI controller                     | Protocol-specific                | No                                |
| Ethernet MAC                       | Protocol-specific                | No                                |
| USB packet engine                  | Mostly protocol-specific         | Usually no                        |
| Programmable timer / cycle counter | Generic timing primitive         | Yes                               |
| Bidirectional serializer / sampler | Generic data-movement primitive  | Yes                               |
| Pin direction / atomic GPIO engine | Generic electrical/I/O primitive | Yes                               |
| Configurable CRC/LFSR engine       | Generic coding/check primitive   | Yes                               |
| Programmable line coder            | Generic symbol/bit transform     | Yes, if sufficiently configurable |
| Edge detector / event FIFO         | Generic receive primitive        | Yes                               |

### 3. Proposed Top-Level Cores / Engines

Recommended starting architecture: one small protocol-control core plus several generic hardware engines. The control core defines protocol behavior; the engines provide deterministic wire-speed operations.



Optional: duplicate small micro-engines for concurrency (e.g. RX and TX).

### 3.1 Protocol Control Core

- Small programmable processor or state-machine ISA optimized for protocol control rather than general computation.
- Responsibilities: branches, loop control, small register file, parameter setup, descriptor issue, protocol state, exception/event handling, and interaction with I/O engines.
- Candidate instructions: SET/CLR/WRITE pins, READ/SAMPLE, WAIT cycles, WAIT edge/level, SHIFT/TRANSFER, LOOP, conditional branch, SETDIR, PUSH/POP FIFO, CRC update/read, and engine configuration.
- The ISA should minimize instructions required between externally visible edges. Predictable latency is more important than rich arithmetic.

### 3.2 Timing and Event Engine

- Provides deterministic cycle timing independent of software instruction sequences.
- Functions: cycle countdown, compare-at-time, periodic events, wait-until-edge/level, timeout, optional timestamp capture, and phase scheduling.
- Should support both internally clocked protocols (UART/SPI master) and externally clocked protocols (SPI slave/JTAG/edge-driven interfaces).
- A key architectural question is whether waits stall the control core or are delegated to an autonomous event engine.

### 3.3 Serializer / Sampler Engine

- Handles repetitive bit-level movement so the control core does not execute one instruction per wire bit.
- Functions: programmable-width shift in/out, MSB/LSB-first selection, simultaneous input/output, configurable bit period/phase, masks, optional parallel pin groups, and handoff to/from FIFOs.
- This engine should be capable of implementing SPI-like transfers, UART bit streams, JTAG scans, and the raw serial bit movement required by USB/Ethernet line coding.

### 3.4 GPIO / Direction Engine

- Owns physical pin state and pin-direction control.
- Functions: atomic multi-pin write, atomic sample, per-pin input/output enable, open-drain behavior, high-Z turnaround, and optional input synchronization / filtering.
- I<sup>2</sup>C and SWD make direction control a first-class operation; USB also benefits from controlled line-state transitions.

### 3.5 Configurable CRC / LFSR Engine

- Provides fast incremental CRC/checksum/LFSR operations without hardcoding Ethernet or USB.

- Parameters should ideally include polynomial, width, initial value, input/output reflection, XOR-out, and bit/byte feed mode.
- This allows the same hardware to serve Ethernet CRC32, USB CRC5/CRC16, CAN CRC variants, scramblers, PRBS generation, and other linear recurrences.

### 3.6 Programmable Line-Coding / Bit-Transform Engine

- Optional but strategically important for the USB/Ethernet stretch goals.
- The block should transform a logical bitstream into physical symbols and/or reverse the operation on receive.
- Useful behaviors include NRZI, Manchester, bit stuffing/de-stuffing, transition detection, invert/XOR-with-state, and small programmable finite-state transforms.
- Avoid fixed “USB decoder” or “Ethernet Manchester decoder” blocks. A small LUT/FSM-driven coder is more aligned with the general-purpose goal.

### 3.7 FIFOs / Scratch State / Host Interface

- Small FIFOs decouple protocol timing from software/control timing and allow blocks to run autonomously for short bursts.
- A compact scratch/register RAM may hold descriptors, outgoing data, captured samples, and protocol state.
- The external programming/control interface should be simple: load program, configure pins/clock, push TX data, read RX/event data, start/stop execution, and inspect state for debugging.

### 3.8 Optional Multiple Micro-Engines

- Multiple very small programmable engines may be more useful than one larger core because protocols are inherently concurrent.
- Examples: one engine tracks external edges while another prepares responses; one performs RX while another performs TX; one services timing-critical wire operations while a supervisory core handles protocol-level state.
- This is conceptually similar to PIO state machines, but the competition opportunity is to choose a more efficient split and ISA for the ASIC area/timing budget.

## 4. Mapping Target Protocols onto the Generic Engines

| Protocol         | Primary primitives                              | Architectural significance                                                                                                                                |
|------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| UART             | Timer + serializer + sampler                    | Start/stop framing and baud timing remain firmware-controlled.                                                                                            |
| SPI              | Serializer/sampler + GPIO + timing              | Clock phase/polarity, chip-select behavior, word width, and transaction structure are software parameters.                                                |
| I <sup>2</sup> C | GPIO direction + edge wait + timer + serializer | Open-drain drive/release, ACK/NACK, clock stretching, arbitration, and transaction state are composed from generic pin/event operations.                  |
| JTAG / SWD       | Serializer + GPIO direction + edge/event logic  | JTAG scans are generic synchronous shifts; SWD stresses bidirectional turnaround.                                                                         |
| Low-speed USB    | Serializer + line coder + CRC + timing + GPIO   | NRZI, bit stuffing, sync/EOP, CRC5/CRC16, line states, and turnaround timing should be composed rather than hidden in a USB block.                        |
| 10 Mb Ethernet   | Serializer + Manchester coder + CRC + timing    | Manchester encoding/decoding, preamble/SFD emission, CRC32, inter-frame timing, and receive edge handling stress throughput and deterministic scheduling. |
| CAN (future)     | Sampler + timing + CRC/LFSR + event logic       | Arbitration, bit timing, stuffing, and CRC provide another strong test of generic primitives.                                                             |

## 5. Recommended Boundary Between Firmware and Hardware

- Firmware should own protocol semantics: packet/transaction state, sequencing, error policy, retry behavior, address interpretation, framing rules, and configuration.
- Hardware engines should own cycle-critical repeated mechanisms: precise waits, shifts, samples, transitions, CRC recurrence, bit transforms, atomic pin changes, and buffering.

- Do not add an Ethernet MAC or USB controller merely to claim stretch-goal support. If a special block is required, it should expose a reusable primitive that plausibly benefits other protocols.
- A useful test: if the block were renamed, would several unrelated protocols still use it naturally? If not, it is probably too protocol-specific.

#### BAD SPLIT

protocol\_cpu + uart\_core + spi\_core + i2c\_core + usb\_core + ethernet\_mac

#### BETTER SPLIT

protocol\_cpu  
 + timing/event engine  
 + bidirectional serializer/sampler  
 + GPIO/direction engine  
 + configurable CRC/LFSR  
 + programmable line coder  
 + FIFOs/scratch storage

Protocols are programs/configurations over the same primitives.

## 6. Candidate Execution Model

A useful starting hypothesis is a two-level execution model:

- Level 1 — control ISA: executes branches, loops, state changes, descriptor setup, and event handling.
- Level 2 — autonomous micro-operations: timing, shifting, sampling, line coding, and CRC proceed for multiple wire cycles after one control instruction.

Example conceptual program for an SPI-like transfer:

```
WRITE_PINS CS=0
CONFIG_XFER width=8, lsb_first=0, period=4, sample_phase=1
XFER tx=r0, rx=r1
WAIT_DONE
WRITE_PINS CS=1
```

The XFER command may consume eight wire bits autonomously. The control core does not need to execute 8 x (write + wait + sample + wait) instruction sequences.

For slower or irregular protocols, software can remain closer to individual edges. For higher-throughput stretch goals, the same generic engines perform longer autonomous operations. This allows one architecture to span “bit-bang-like” control and hardware-assisted streaming without introducing fixed protocol blocks.

## 7. Architectural Questions for the Planning Agent

- What is the minimum ISA that can express UART, SPI, and I<sup>2</sup>C cleanly without excessive instruction count?
- What is the worst-case pin-to-pin reaction latency: input edge observed → branch/decision → output change?
- Which operations must complete in one cycle, and which can be multi-cycle without harming protocol timing?
- At the target process and realistic clock frequency, can low-speed USB and 10BASE-T be handled one bit per core cycle, or is an autonomous serializer/line-coder mandatory?
- Would two or four tiny micro-engines provide better protocol concurrency per unit area than one larger core?
- Should the line-coding engine be a fixed menu of generic transforms (NRZI/Manchester/stuffing) or a tiny programmable LUT/FSM?
- How configurable should the CRC/LFSR unit be before configuration logic costs more area than several optimized datapaths?
- How are external asynchronous inputs synchronized without destroying edge timing? Is oversampling required for some protocols?
- What FIFO depths are sufficient to isolate protocol timing while remaining area-efficient in the 8x4 Tiny Tapeout budget?
- What memory technology and program size are practical in IHP 130 nm: SRAM macro vs registers/latches?
- How should pin ownership and contention work when multiple micro-engines share the same I/O bank?
- What mechanisms support open-drain, tri-state turnaround, and differential/logical line states?
- How are programs loaded and debugged on real silicon? What observability counters, trace FIFO, single-step, or state readback are worth the area?

- What is the verification strategy: instruction-level model, protocol reference models, randomized timing, formal properties, and FPGA prototyping?
- Which architecture metrics should drive comparison: area, maximum clock, edge-to-response latency, minimum bit period, code size per protocol, energy, and number of concurrent protocol machines?

## 8. Suggested First Architecture Study

- Build a minimal cycle-accurate architectural simulator before committing to RTL. Model the ISA, pin timing, engine latencies, and FIFO behavior.
- Write reference firmware for UART TX/RX, SPI master/slave, and I<sup>2</sup>C master/slave. Record instruction count, minimum bit period, and reaction latency.
- Add USB low-speed transmit/receive as the first stress case. Determine which generic primitives are missing rather than immediately adding USB-specific logic.
- Add 10BASE-T transmit/receive as a second stress case. Evaluate whether Manchester and receive edge recovery fit a general line-coding/event architecture.
- Compare one-core, multi-microengine, and two-level control+I/O-engine organizations under a rough synthesized area budget.
- Only after the execution model is stable, write RTL and begin early synthesis/place-and-route experiments. The competition guidance explicitly recommends synthesizing early because mapped area and routability can differ substantially from RTL intuition.

## 9. Proposed Success Criteria

| Criterion         | Meaning                                                                                                                                                             |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Generality        | A new simple serial protocol can be implemented by firmware/configuration without modifying RTL.                                                                    |
| Baseline coverage | UART, SPI, and I <sup>2</sup> C work in both useful controller/device modes where practical.                                                                        |
| Stretch coverage  | Low-speed USB and 10 Mb Ethernet are demonstrably achievable using the same primitives, with minimal protocol-specific hardwiring.                                  |
| Determinism       | External edge timing and input-to-output response latency are predictable and documented.                                                                           |
| Area efficiency   | The design fits comfortably within the competition allocation with margin for routing and clock-tree overhead.                                                      |
| Verification      | A software reference model and protocol tests validate both ISA behavior and wire-level timing; formal checks are applied to critical control/FIFO/handshake logic. |
| Debuggability     | Programs, engine state, captured events, and error conditions are observable enough to make first-silicon bring-up realistic.                                       |

## 10. Bottom-Line Architecture Hypothesis

**Recommended initial hypothesis:** a small deterministic protocol-control ISA controlling one or more lightweight I/O micro-engines, with dedicated but protocol-agnostic timing/event, serializer/sampler, GPIO/direction, configurable CRC/LFSR, and programmable line-coding facilities. Add small FIFOs/scratch memory for decoupling and observability. Treat USB and Ethernet as stress tests of the generic primitives, not as justification for fixed protocol controllers.

**The architecture should be judged by how much protocol behavior remains programmable while still achieving deterministic wire timing.**

**Source context:** Jane Street, “Can you design a chip? Announcing the protocol emulator ASIC competition,” September 10, 2026. General-purpose protocol emulator ASIC; baseline UART/SPI/I<sup>2</sup>C; stretch goals low-speed USB and 10 Mbit Ethernet; IHP 130 nm CMOS5L through Tiny Tapeout; 8x4 tile maximum planned allocation; deadline January 18, 2027. <https://blog.janestreet.com/protocol-emulator-asic-competition/>